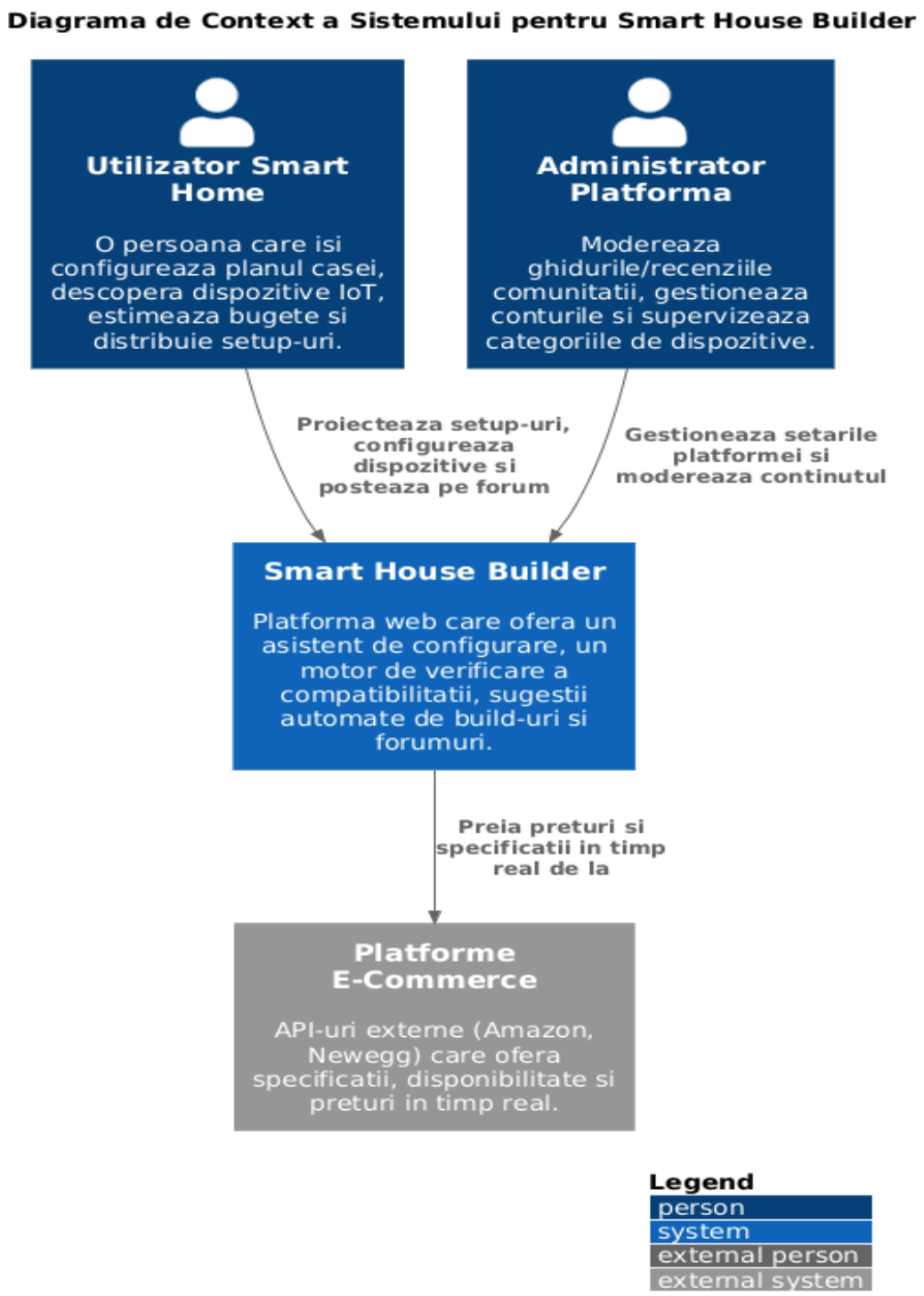
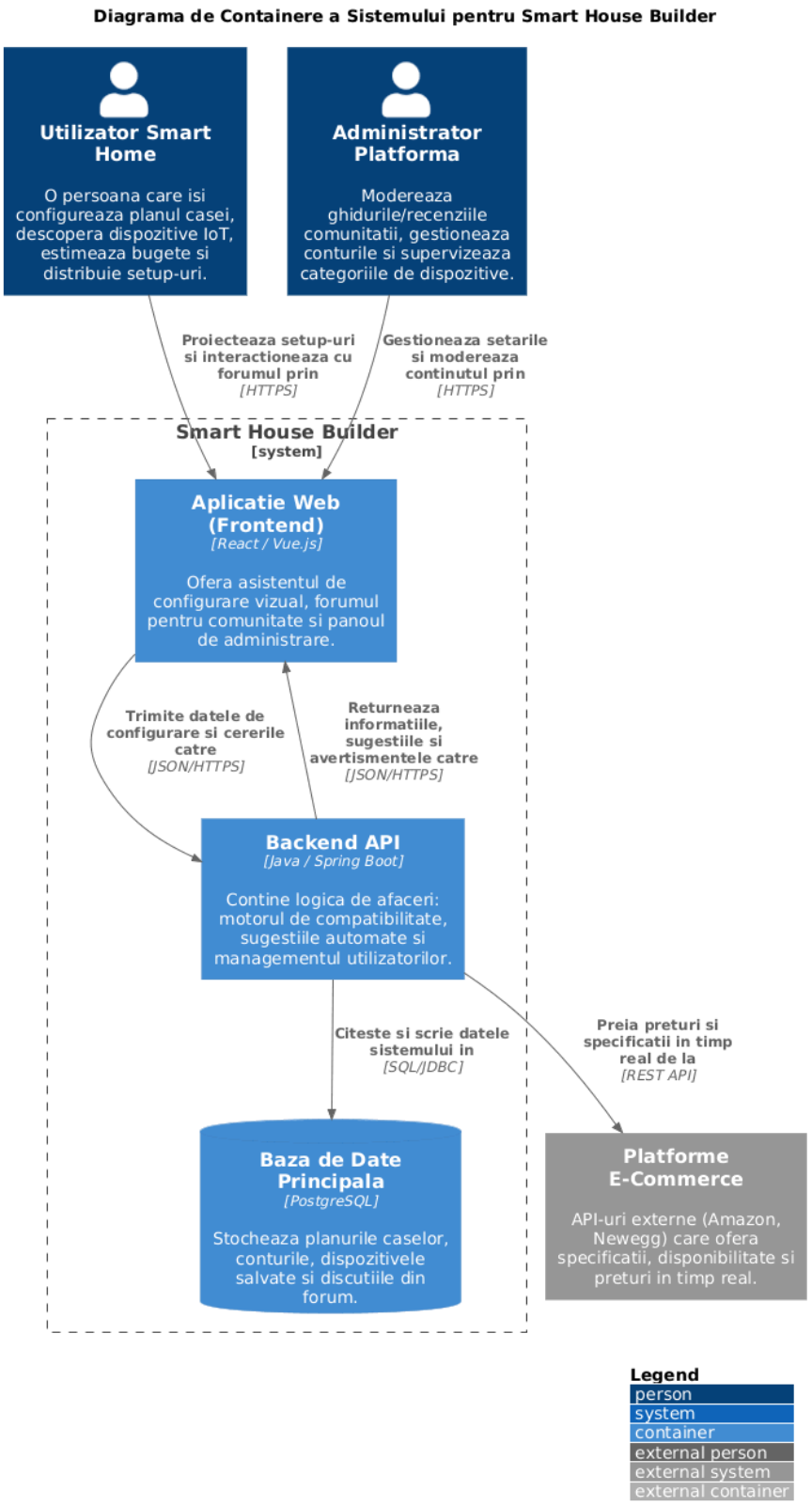
# Smart House Builder

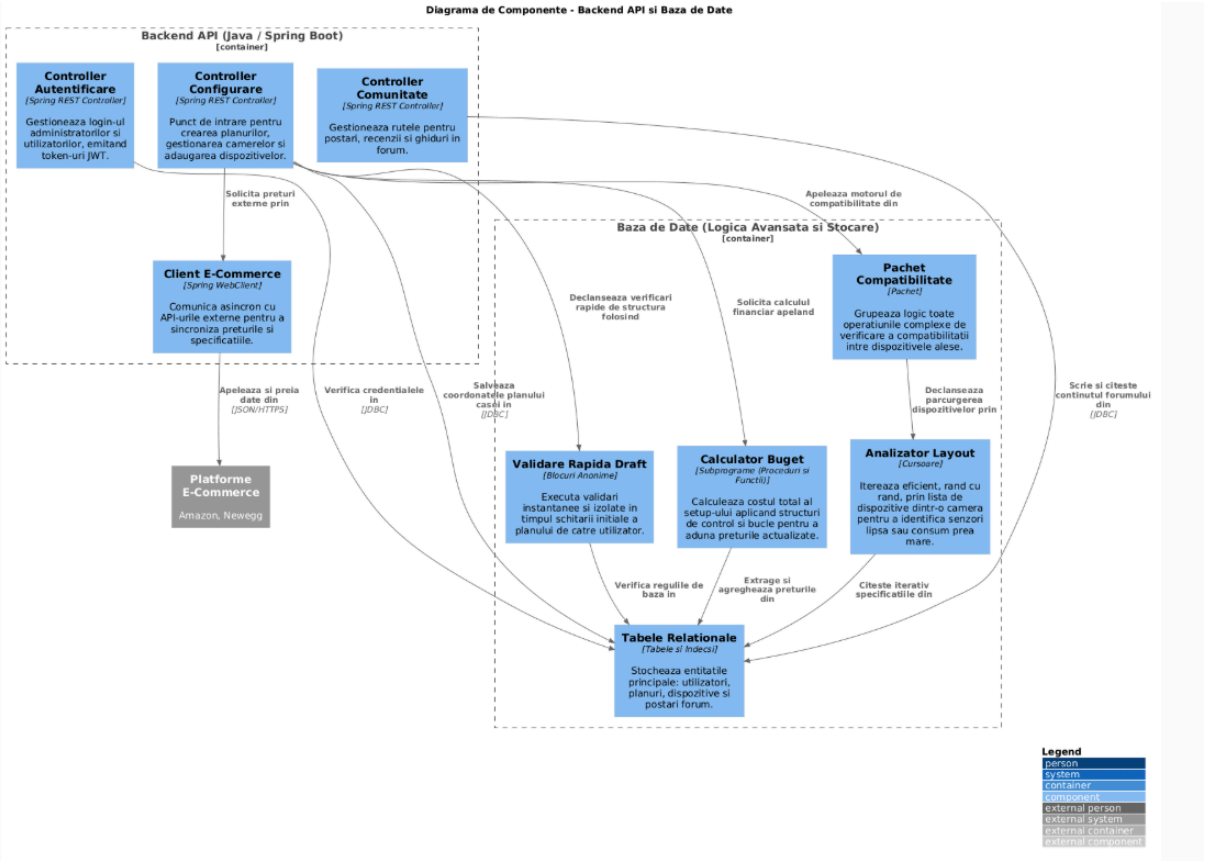
## Diagrama C1 pentru Smart house



# C2 Diagrama pentru container



# C3 Component Diagram




# 3)Clase fara AI

```java
public class SmartComponent {
    protected String id;
    protected String name;
    protected double basePrice;


    public boolean validateDraftStatus() {
        System.out.println("Validare pentru: " + name);
        return id != null;
    }


    public double calculateFinalPrice(double taxRate) {
        return basePrice + (basePrice * taxRate);
    }
}

public class SmartDevice extends SmartComponent {
    private String protocol; // ex wi fi
    private double powerConsumption;


    public boolean checkCompatibility(SmartDevice otherDevice) {
        if (this.protocol.equalsIgnoreCase(otherDevice.protocol)) {
            System.out.println("Verific compatibilitate dintre  " +
this.name + " si " + otherDevice.name);
            return true;
        }
        return false;
    }


    public boolean exceedsPowerLimit(double threshold) {
        return powerConsumption > threshold;
    }
}
import java.util.List;
import java.util.ArrayList;

public class RoomLayoutManager {
    private List<SmartDevice> devices = new ArrayList<>();
```

```
    public List<String> analyzeMissingSensors() {
        List<String> alerts = new ArrayList<>();
        boolean hasMotionSensor = false;

        for (SmartDevice device : devices) {
            if (device.name.toLowerCase().contains("miscare ")) {
                hasMotionSensor = true;
                break;
            }
        }

        if (!hasMotionSensor) {
            alerts.add("Nu s a identificat niciun senzor de miscare");
        }
        return alerts;
    }


    public double calculateTotalSetupBudget(double discountCode) {
        return devices.stream()
                .mapToDouble(d -> d.calculateFinalPrice(0.19)) // ex
tva 19%
                .sum() - discountCode;
    }

    public void addDevice(SmartDevice device) {
        if(device.validateDraftStatus()) {
            devices.add(device);
        }
    }
}
```

# 4)Clasele cu AI

```
// ==========================================
// CONTROLLERS (Spring Boot)
// ==========================================

import org.springframework.web.bind.annotation.*;
import org.springframework.http.ResponseEntity;
```

```java
import org.springframework.stereotype.Component;
import org.springframework.web.reactive.function.client.WebClient;

// ==========================================
// CONTROLLER AUTENTIFICARE
// ==========================================
@RestController
@RequestMapping("/api/auth")
class ControllerAutentificare {

   @PostMapping("/login")
   public ResponseEntity<String> login(@RequestParam String username,
                                       @RequestParam String password) {
       // Simulare verificare credentiale (fara DB)
       return ResponseEntity.ok("jwt-token-simulare");
   }
}

// ==========================================
// CONTROLLER CONFIGURARE
// ==========================================
@RestController
@RequestMapping("/api/setup")
class ControllerConfigurare {

   private final ValidareRapidaDraft validareRapidaDraft;
   private final PachetCompatibilitate pachetCompatibilitate;
   private final CalculatorBuget calculatorBuget;
   private final EcommerceClient ecommerceClient;

   public ControllerConfigurare(ValidareRapidaDraft validareRapidaDraft,
                                PachetCompatibilitate
pachetCompatibilitate,
                                CalculatorBuget calculatorBuget,
                                EcommerceClient ecommerceClient) {
       this.validareRapidaDraft = validareRapidaDraft;
       this.pachetCompatibilitate = pachetCompatibilitate;
       this.calculatorBuget = calculatorBuget;
       this.ecommerceClient = ecommerceClient;
   }

   @PostMapping("/plan")
   public ResponseEntity<String> creeazaPlan(@RequestBody String planJson)
{
       validareRapidaDraft.valideazaDraft(planJson);
       return ResponseEntity.ok("Plan salvat (simulare).");
   }

   @PostMapping("/compatibility-check")
   public ResponseEntity<String> verificaCompatibilitate(@RequestBody
String setupJson) {
```

```
        return
ResponseEntity.ok(pachetCompatibilitate.verificaCompatibilitate(setupJson))
;
    }

    @PostMapping("/budget")
    public ResponseEntity<Double> calculeazaBuget(@RequestBody String
setupJson) {
        return
ResponseEntity.ok(calculatorBuget.calculeazaBuget(setupJson));
    }

    @GetMapping("/prices-sync")
    public ResponseEntity<String> sincronizeazaPreturi() {
        ecommerceClient.sincronizeazaPreturi();
        return ResponseEntity.ok("Preturi sincronizate (simulare).");
    }
}

// ==========================================
// CONTROLLER COMUNITATE
// ==========================================
@RestController
@RequestMapping("/api/community")
class ControllerComunitate {

    @PostMapping("/post")
    public ResponseEntity<String> creeazaPostare(@RequestBody String
postJson) {
        return ResponseEntity.ok("Postare creata (simulare).");
    }

    @GetMapping("/posts")
    public ResponseEntity<String> listaPostari() {
        return ResponseEntity.ok("Lista postari (simulare).");
    }
}

// ==========================================
// CLIENT E-COMMERCE (API extern)
// ==========================================
@Component
class EcommerceClient {

    private final WebClient webClient;

    public EcommerceClient(WebClient.Builder builder) {
        this.webClient =
builder.baseUrl("https://api.fake-ecommerce.com").build();
    }

    public void sincronizeazaPreturi() {
```

```
        System.out.println("Sincronizare preturi cu Amazon/Newegg
(simulare)...");
    }
}

// ==========================================
// LOGICA AVANSATA - VALIDARE RAPIDA
// ==========================================
@Component
class ValidareRapidaDraft {

    public void valideazaDraft(String planJson) {
        System.out.println("Validare rapida a planului (simulare)...");
    }
}

// ==========================================
// LOGICA AVANSATA - ANALIZATOR LAYOUT
// ==========================================
@Component
class AnalizatorLayout {

    public void analizeazaLayout(String setupJson) {
        System.out.println("Analiza layout (cursoare simulate)...");
    }
}

// ==========================================
// LOGICA AVANSATA - PACHET COMPATIBILITATE
// ==========================================
@Component
class PachetCompatibilitate {

    private final AnalizatorLayout analizatorLayout;

    public PachetCompatibilitate(AnalizatorLayout analizatorLayout) {
        this.analizatorLayout = analizatorLayout;
    }

    public String verificaCompatibilitate(String setupJson) {
        analizatorLayout.analizeazaLayout(setupJson);
        return "Raport compatibilitate (simulare).";
    }
}

// ==========================================
// LOGICA AVANSATA - CALCULATOR BUGET
// ==========================================
@Component
class CalculatorBuget {

    public double calculeazaBuget(String setupJson) {
        return 999.99; // Simulare
```

```
    }
}
```

**AI-uri folosite: Microsoft copilot, pentru construirea claselor de backend API:[https://copilot.microsoft.com/conversations/join/qbybBkZwDDfDahEFYJH1k](https://copilot.microsoft.com/conversations/join/qbybBkZwDDfDahEFYJH1k)**